

Lab 2

Localization using Turtlebot4 and Extended Kalman Filter

Emilio Rivas

TA: Georgia Kouvoutsakis

SID: 862302047

4/30/26

Background Description:

A robot needs to know where it is. Its position (x, y) and heading θ . Wheel encoders can estimate this by counting rotations (this is **odometry**, the `/odom` topic), but small errors accumulate over time. After a minute of driving, the estimate might be off by half a meter. This is **drift**, and it's why the red dashed line on your plot peels away from the green ground truth.

The fix

We give the robot a LIDAR and a known map of three beacons. Every time the LIDAR sees one, the robot can use that observation to correct its drifted estimate. The challenge: how do you blend noisy odometry with noisy LIDAR readings intelligently?

The Kalman filter

A Kalman filter tracks two things:

- μ — best guess of (x, y, θ)
- Σ — uncertainty about that guess

It runs a two-step loop:

Predict (when odometry arrives): push the estimate forward using the motion model, and grow Σ because moving without checking the world adds drift.

Update (when the LIDAR sees a beacon): compare the expected measurement to the actual one, nudge μ toward the truth, and shrink Σ because we just got new information. The **Kalman gain** decides how much weight to give the measurement vs. the existing estimate.

Why "Extended"?

The robot's motion and measurement equations are nonlinear (they involve `sin`, `cos`, `sqrt`, `atan2`). The EKF handles this by **linearizing** at each step. That's where the Jacobians `F_x`, `F_u`, and `H` come in. Those are the 9 TODO blocks in `ekf.py`:

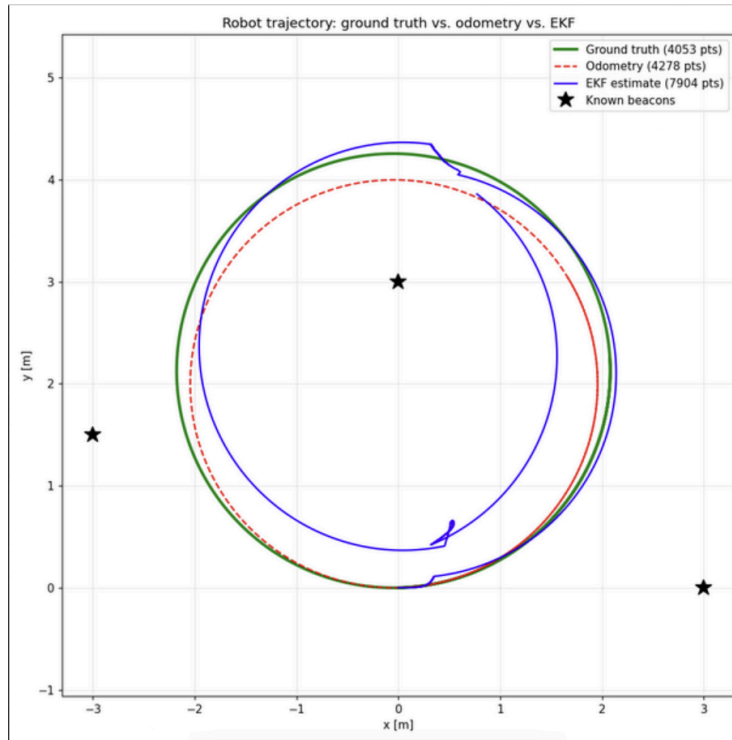
- TODOs 1–3: predict step
- TODOs 4–9: update step

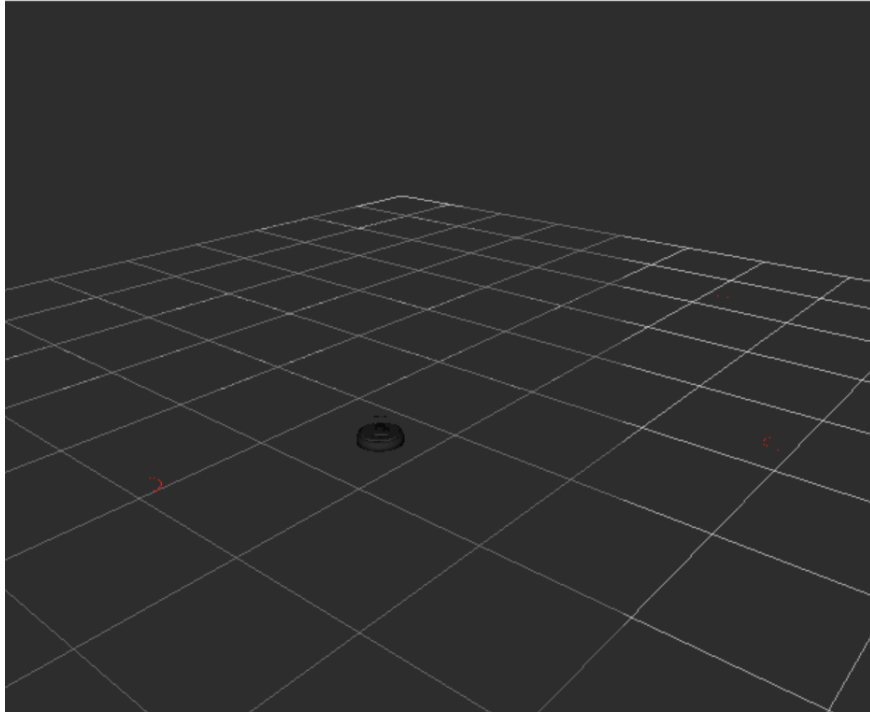
The result

Three lines on your trajectory plot tell the whole story:

- **Green** — ground truth (the reference)
- **Red dashed** — odometry alone (drifts over time)
- **Blue** — the EKF estimate (hugs the green line, snapping back into alignment whenever a beacon is seen)

Simulation results:





Applications: Lab 2 - Localization usi... Gazebo Sim RViz* Xfce Terminal 2026-04-29 21:05 Ubuntu

File Panels Help

Interact Move Camera Select Focus Camera Measure 2D Pose Estimate 2D Goal Pose Publish Point

Displays

Selectable	☒
Style	Flat Squares
Size (m)	0.01
Size (Pixels)	3
Alpha	1
Decay Time	0
Position Transformer	XYZ
Color Transformer	Intensity
Axis	Z
Autocompute Value...	☒
Use Fixed Frame	☒
Channel Name	Intensity
Use rainbow	☒
Invert Rainbow	☐
Min Color	0; 0; 0
Max Color	255; 255; 255
Autocompute Inte...	☒
Min Intensity	0
Max Intensity	0
Color	255; 255; 255

Depth
Set the depth of the incoming message queue. Increasing this is useful if your incoming TF data is delayed significantly from your message data, but it can greatly increase memory usage if the messages

Add Duplicate Remove Rename

Views

Type: Orbit (rviz_default.j) Zero

Current View

Near Clip Di...	0.01
Enable Ster...	☐
Invert Z Axis	☐
Target Frame	<Fixed Frame>
Distance	3.40372
Focal Shape...	0.05
Focal Shape...	☒
Yaw	0.507116
Pitch	0.637641
Focal Point	4.7915; 3.9511; 2....

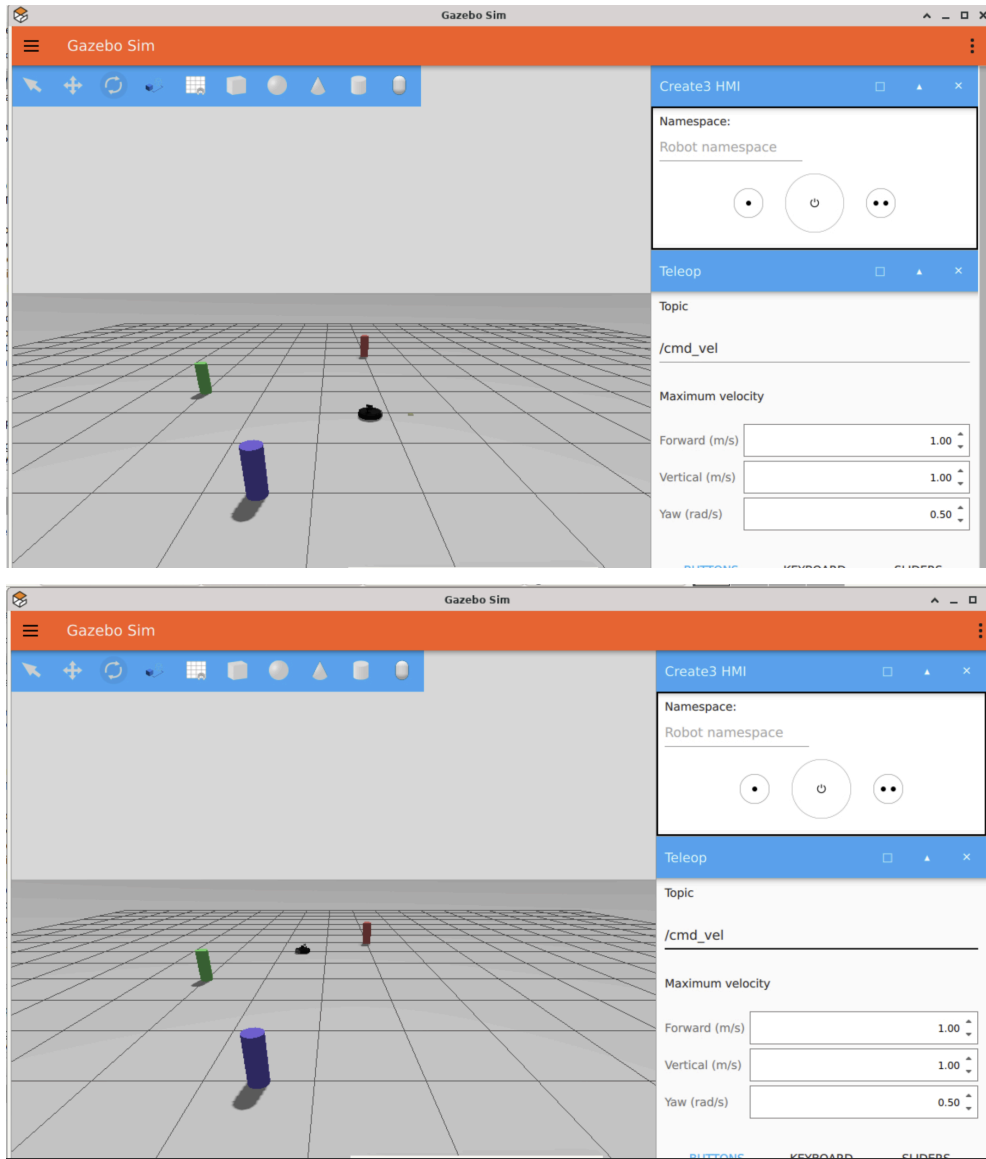
Save Remove Rename

Time

ROS Time: 514.13 ROS Elapsed: 514.13 Wall Time: 1777521940.08 Wall Elapsed: 2513.15

Reset

Experimental 31 fps



Discussion:

Did the EKF estimate stay closer to ground truth than the odometry estimate?

Yes, the green circle with radius about 2 m is centered at (0, 2). The blue EKF line lies, for most of the trajectory, on top of green, meaning that the two lines are - for the right and upper segments of the circle - congruent. The line for the red dashed odometry, on the other hand, describes a smaller circle, and is pulled inward and shifted to the right. The spacing between red and green describes the drift, and is, for example, larger than the spacing between blue and green. The EKF is performing the needed updates by using the LIDAR-to-beacon updates to correct the drift of the odometry.

When during the trajectory was the difference between odometry and EKF largest?

In the initial segment of the path (around (0.5, 0.5)), and then again at (0.7, 4.3), the blue line has jumps. These jumps show some of the more dramatic impacts the Extended Kalman filter (EKF) correction has. Because the undulation of the EKF is coupled to the odometry, there will only be jumps when the odometry and EKF predictions differ. The line jumps show some of the highest discrepancies. Over time, the EKF is pushed more toward the beacon's truth values. The updates drive the EKF line toward the beacon. Contrarily, odometry (the red line) can only be pushed inward through the loss of truth.

In this case, the line explains the impacts of odometry drift best. At the end of the experiment, odometry drift has happened the most, causing the lines to diverge, while the EKF line stays smooth due to the updates.

What does the covariance ellipse around the EKF estimate represent? (Hint: look at the Sigma matrix in your code — what do its diagonal entries tell you?)

The Sigma matrix in your code represents the EKF's level of uncertainty regarding the state of the system, captured as (x, y, θ). It is represented as a 3x3 matrix, and the level of uncertainty in each dimension is captured in the diagonal entries. For example:

Sigma[0,0]: variance in x

(None, corresponds to how uncertain the filter is in this dimension about the robot's position in x)

Sigma[1,1]: variance in y

(None, corresponds to uncertainty in robot's position in y)

Sigma[2,2]: variance in θ

(None, corresponds to uncertainty in the heading)

The off-diagonal entries exemplify the relationships and correlations of uncertainty in each dimension (i.e. how the filter's uncertainty about heading also introduces coupling errors in the other x and y dimensions).

If you draw a 2D covariance ellipse using just the (x, y) block of Sigma, this represents a confidence region. A stated approximation of the ellipse is the 2σ , or the 95% confidence ellipse. This latter means that the true position of system has a 95% chance of being in the region bounded by this ellipse.

The ellipse's response to the two parts of the EKF run is as follows:

The Predict step corresponds to each successive odometry message ($F_u @ M @ F_u.T$) and causes the covariance ellipse to grow. It is this delta where the filter's level of certainty, or uncertainty about a state grows.

The Update step corresponds to each successive successful LIDAR-to-beacon match and causes the covariance ellipse to shrink. This is the delta where the filter's level of uncertainty, or certainty about a state, improves.

Therefore, during the run, the covariance ellipse actually 'breathes'. A wider ellipse corresponds to a more uncertain cone of focus, and a narrower ellipse corresponds to a more certainty cone of focus.